Homework – Variables and NASM

Goal:

Write the back-end of the compiler by implementing functioning NASM code.

Purposes:

- Previously, parsed input was processed into a binary tree and/or post-order stack. Now you should turn the post-order stack into actual NASM code.
- Support four basic arithmetic operators, namely addition, subtraction, multiplication, and division.
- Compilers aren't very exciting without an operator = assignment. Add this functionality.
- Utilize a symbol table to track a variable's type and where it is stored in the program stack.

Instructions:

Your overall task is to accept everything in the file `accept.txt`, and create correct NASM code which creates a correct program.

Your other overall task is to reject everything wrong in the file `reject.txt`.

The assignment adds some new syntax. You will need new productions for the assignment operator = as well as a unary operator << to handle print.

For all lines in `accept.txt`, translate all instructions into NASM output covered in prior assignments. In other words, consider these lines:

```
num var8 = 3*4
<< var8
```

That should create NASM code which will ultimately multiply 3 with 4 and store it in a stack space reserved for `var8`. (Previously, you optimized your IR. Unfortunately, if this assignment allowed you to continue to optimize, you could optimize your compiler down to simply print statements. In class I suggested an approach which upon further reflection was a mess, and so that approach won't be used. You must run unoptimized NASM instructions.)

Ensure the << operator actually prints to the console. Floating point values can be limited to 3 digits after the decimal.

All variables should have a symbol table entry that relates it to an offset on the program stack.

Allow ints to be 32 bit.

Do not worry about adding additional features that the files do not cover

Productions:

```
0.     Goal                  -> LineFull
1.     LineFull              -> Decl LineVarNameRemaining
2.     LineFull              -> name AfterName
3.     LineFull              -> <<  GTermSign
4.     AfterName             -> LineVarNameRemaining
5.     AfterName             -> Expr
6.     Decl                  -> int32 NameEnd
7.     Decl                  -> float32 NameEnd
8.     NameEnd               -> name
9.     LineVarNameRemaining  -> =  Expr
10.     Expr                  -> LTermAddSub  AddSub'
11.     LTermAddSub           -> LTermMultDiv  MultDiv'
12.     LTermMultDiv          -> LTermPower  Power'
13.     RTermAddSub           -> RTermMultDiv  MultDiv'
14.     RTermMultDiv          -> RTermPower  Power'
15.     AddSub'               -> +  RTermAddSub  AddSub'
16.     AddSub'               -> -  RTermAddSub  AddSub'
17.     AddSub'               -> ϵ
18.     MultDiv'              -> MultDivAndRightOp
19.     MultDiv'              -> ϵ
20.     MultDivAndRightOp     -> *  RTermMultDiv  MultDiv'
21.     MultDivAndRightOp     -> /  RTermMultDiv  MultDiv'
22.     Power'                -> PowerAndRightOp
23.     Power'                -> ϵ
24.     PowerAndRightOp       -> ^  RTermPower  Power'
25.     LTermPower            -> GTermSign
26.     RTermPower            -> GTermSign
27.     GTermSign             -> GTerm
28.     GTermSign             -> - GTerm
29.     GTerm                 -> Parens
30.     GTerm                 -> name
31.     GTerm                 -> num
32.     Parens                -> (  Expr  )
```

Disallowed:

- You cannot write an ad-hoc compiler. You must retain your LL(1) parser and its productions. You must also retain a binary tree and/or post-order stack.
- You cannot program your code to reject any lines if it has the word ‘bad’ in it. Students who do will receive unspeakable punishments.
- You can change any “<< var8” lines to “print(var8)” if you would like. Just update your productions to match.

Verification:

- Run your compiler against the entire `accept.txt` file and generate a correct NASM file. Verify the assembly is accurate (optimizations still occur where they should). That NASM file should then be compiled and executed with all outputs displayed.
- Run your compiler against the entire `reject.txt` file. It should generate an error on each rejected line. You don’t need to specify why the line was rejected if you don’t want to.
- Like prior assignments, a simple 30-60 second video demonstrating your program is required.

Other notes:

For those who use the process to jump straight to a binary tree in one step, below are the latest steps.

This process is not as pretty as the other two, but does allow for a quick jump to a binary tree containing only terminals of interest and removing unwanted nonterminals along the way.

Your code must add extra knowledge onto productions. For all operators, insert a DONE term immediately after it and then a PLACEHOLDER term after the nonterminal after that. For example, suppose a production is `Alice -> + Bob Carl`. Here the `+` is an operator, and `Bob` and `Carl` are regular nonterminals. Insert both a PLACEHOLDER and DONE so that it reads `Alice -> + PLACEHOLDER Bob DONE Carl`.

In the LL(1) parser, add the following logic to the existing LL(1) parsing algorithm:

- If a non-operator value terminal (such as a number or a variable name) would be consumed off the LL(1) stack, one of two options will occur:
    - If this is the first node of the output tree, create a node and put that token in it. Make this node the current focus node.
    - If the current focus node contains a PLACEHOLDER value, overwrite its value with the current token. Keep the current focus on this node.

- Else if a unary operator would be consumed off the stack
    - If this is the first node of the output tree, create a node and put that token in it. Make this node the current focus node.

- Else if an operator would be consumed off the stack, then create a node to hold the operator token. Swap the current node with the newly created node (e.g. if a parent pointed to the current node, it now points to the new node, otherwise if the node was root then create an operator node at root). Have the newly created node's left link point to the current node. Make this operator node the current focus node.

- Else if a PLACEHOLDER item is at the top of the stack, then create a node as a new right child. That node will contain as data PLACEHOLDER. This node's data will act as a placeholder to be overwritten later. Make this node the current focus node. Pop PLACEHOLDER off the production stack.

- If a sentinel DONE item is the at top of the stack, then this represents that a prior right operand branch has finished. Pop DONE off the production stack. Send the current node focus up to the parent.

- Else if none of prior bullet points matched (such as a standard nonterminal production or parenthesis for grouping), let the existing algorithm proceed as normal.

NASM examples:

```
; How to build
; nasm -felf64 hello.asm
; ld hello.o -o hello.x
; ./hello.x

        section .text
```

```
        global _start

_start:
        mov     eax, 1              ; system call for write
        mov     edi, 1              ; file handle 1 is stdout
        mov     esi, message        ; address of string to output
        mov     edx, 13             ; number of bytes
        syscall                     ; invokes the operating system to do a write
        mov     eax, 60             ; system call for exit
        xor     edi, edi            ; exit code 0
        syscall                     ; invoke operating system to exit

        section .data
message: db     "Hello, World", 10  ; The 10 is \n
```

```
; Compile with:
; nasm -felf64 triangle.asm
; ld triangle.o -entry main

            section     .text
            global      main
main:
            mov         rdx, output     ; rdx holds address of next byte to write
            mov         r8, 1           ; initial line length
            mov         r9, 0           ; number of stars written on line so far
line:
            mov         byte [rdx], '*' ; write single star to buffer
            inc         rdx             ; advance pointer to next cell to write
            inc         r9              ; "count" of stars written so far
            cmp         r9, r8          ; did we reach the number of stars for this line?
            jne         line            ; if not done, keep writing this line
lineDone:
            mov         byte [rdx], 10  ; write a new line char
            inc         rdx             ; advance pointer to next slot in buffer
            inc         r8              ; next line will be one char longer
            mov         r9, 0           ; reset count of stars written on this line
            cmp         r8, maxlines    ; wait, did we finish the last line?
            jng         line            ; if not, begin writing this line
done:
            mov         rax, 1
            mov         rdi, 1
            mov         rsi, output
            mov         rdx, dataSize
            syscall                     ; write the triangle on out

            mov         rax, 60
            xor         rdi, rdi
            syscall                     ; exit cleanly

            section     .bss

maxlines    equ         8
dataSize    equ         44
output:     resb        dataSize
```

```
; ****************************************
; ************** printf usage **************
```

```
; ****************************************
; nasm -felf64 printf.asm -o printf.o
; gcc -m64 -o printf.x printf.o -lc -no-pie
; ./printf.x
; Example output
; Hello, this is my string
; 42
; 123 (you have to type this in and hit enter)
; 123
; 3.14 (you have to type this in and hit enter)
; 3.140000

            section .data
msg:        db "Hello, this is my string", 0
fmtstr:     db "%s", 10, 0
fmtuint:    db "%d", 10, 0
fmtfloat:   db "%f", 10, 0
fmtuintin:  db "%d", 0
fmtfloatin: db "%f", 0
float1:     dd     0.0

            section .bss
var1:       resd   1

            section .text

            extern printf
            extern scanf
            global main
main:
            push rbp ; Push base pointer onto stack to save it

            ; print a string
            mov esi, msg
            mov edi, fmtstr
            mov eax, 0
            call printf

            ; print an unsigned integer
            mov esi, 42
            mov edi, fmtuint
            mov eax, 0
            call printf

            ; read an integer
            lea edi, [fmtuintin]
            lea esi, [var1]
            mov eax, 0
            call scanf

            ; print integer back out
            lea edi, [fmtuint]
            mov esi, [var1]
            xor eax, eax
            call printf

            ; read float
            lea edi, [fmtfloatin]  ; read string format input
            lea esi, [float1]      ; read float storage location
            call scanf
```

```
; print float
lea edi, [fmtfloat]    ; read string format output
movss xmm0, [float1]   ; read float storage location into xmm0
cvtss2sd  xmm0, xmm0
mov eax, 1             ; tell printf there is 1 float parameter
call printf

pop rbp  ; restore stack base pointer

mov         rax, 60
xor         rdi, rdi
syscall                    ; exit cleanly
```